MAREVLO · PRODUCTION RAG

Embeddings

The "meaning fingerprint," finally explained — how a computer turns words into something it can compare.

PART I — FOUNDATIONS

Module 3

Production RAG: Building Retrieval-Augmented Generation Systems That Survive Real Users

What this module is about

All through Module 2 we kept mentioning a "meaning fingerprint" — the thing that lets your system find the right piece of text. We promised to explain it properly. This is that module. The fingerprint has a real name: an **embedding**. By the end of these pages you will know exactly what an embedding is, why it lets a computer measure how similar two pieces of text are, how the size of an embedding affects cost and quality, and how to choose the tool that makes them.

This is a foundational idea. Almost everything in the rest of the course — searching, ranking, debugging, scaling — rests on understanding embeddings well. The good news is that the core idea is genuinely simple once you see it, and we are going to build it up from zero, with pictures, so that it clicks.

> **THE WHOLE MODULE IN ONE SENTENCE**
>
> An embedding turns a piece of text into a list of numbers that act like map coordinates for its meaning — so that texts which mean similar things end up close together, and "find the relevant text" becomes "find the nearest points."

A reminder about the code and where we are

As in Module 2, every code example is written from scratch around our own running example — the little coffee-machine manual — and uses real, public libraries whose exact import paths you should always check against the version you install. We are still inside the indexing pipeline. Module 2 cut the document into pieces; this module turns each piece into an embedding; Module 4 will store those embeddings so they can be searched.

```
# The indexing pipeline:
Load  -->  Chunk  -->                    -->  Store

# Each chunk from Module 2 becomes one embedding here.
```

SECTION 1

What an embedding actually is

Start here. An embedding is just a list of numbers — but the numbers are placed so cleverly that they capture meaning.

Here is the problem an embedding solves. A computer cannot compare the *meaning* of two sentences directly. It has no instinct that "How do I clean the machine?" and "Wipe the parts after each use" are related. To a computer, those are just different strings of characters. We need a way to turn meaning into something a computer is very good at: numbers it can compare.

An **embedding model** does exactly this. You hand it a piece of text, and it hands back a list of numbers — perhaps a few hundred of them, perhaps a few thousand. That list is the embedding. The magic is not that it produces numbers; anything can produce numbers. The magic is *which* numbers it produces: the model is trained so that pieces of text with similar meaning come out with similar lists of numbers, and pieces with different meaning come out with very different lists.

ANALOGY — COORDINATES ON A MAP

Think of a map of a country. Every town has coordinates — two numbers, a latitude and a longitude — that place it somewhere on the map. Towns that are near each other have similar coordinates. An embedding does the same thing for meaning: it gives every piece of text its own coordinates on a giant "meaning map," and pieces that mean similar things land near each other. The only difference is that the meaning map has far more than two coordinates, for reasons we will get to in Section 3.

Let us look at the picture, because this is the idea that makes everything else make sense.

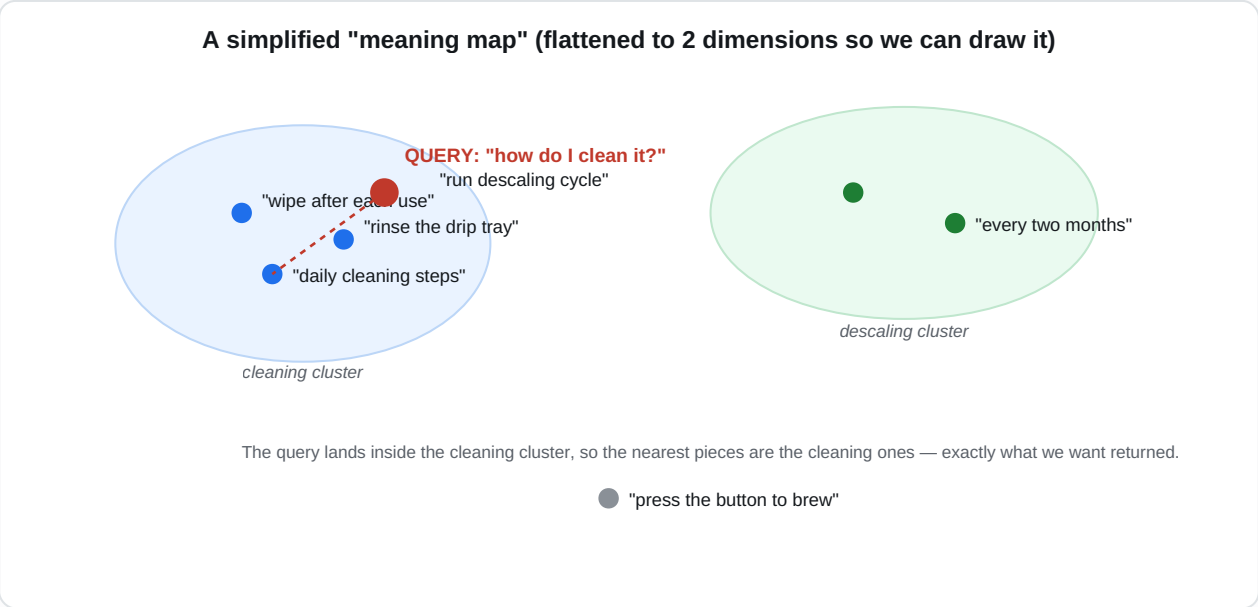


Figure 1. Pieces about the same topic cluster together. A question lands near its topic, so "find the answer" becomes "find the nearest points." (Real maps have hundreds of dimensions; we flattened it to two so it fits on a page.)

That is the whole idea in one picture. The embedding model has placed each piece of the manual somewhere on the meaning map. The cleaning pieces cluster in one area, the descaling pieces in another, and the lone brewing instruction sits off by itself. When a user asks "how do I clean it?", we embed that question too — it lands right inside the cleaning cluster — and we simply return the nearest pieces. No keyword matching, no guessing; just nearness on the meaning map.

SECTION 2

How "similar" becomes "close"

Once every text is a point in space, similarity becomes a measurable distance. Here is the simple maths, explained gently.

If every piece of text is a point on the meaning map, then "which pieces are most similar to this question?" turns into "which points are closest to this question's point?" That is something a computer can answer instantly and precisely. But we need to agree on what "close" means, and there are two common ways to measure it.

The first way is plain **distance** — literally how far apart two points are, like measuring with a ruler. The second, and the one used most often for text, is called **cosine similarity**, which measures the *angle* between two points seen as arrows from the centre. If two arrows point in nearly the same direction, they are very similar; if they point in very different directions, they are not.

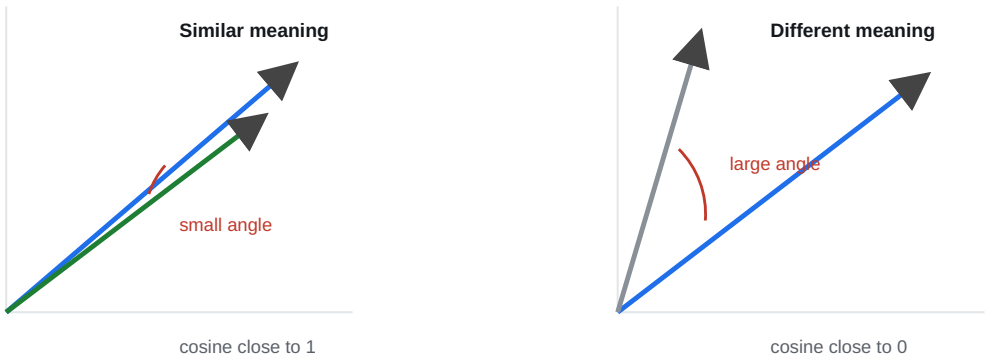


Figure 2. Cosine similarity measures the angle between two pieces of text seen as arrows. A small angle means similar meaning (score near 1); a wide angle means unrelated (score near 0).

You do not need to do this maths by hand — the libraries do it for you — but seeing it once removes the mystery. Cosine similarity gives a score, usually between 0 and 1 for text, where a higher score means more similar. A score near 1 means "these two pieces mean almost the same thing," and a score near 0 means "these two are basically unrelated." Below is our own small function that computes it, written out so you can see there is nothing hidden inside.

LAB 3.1 — Cosine similarity, written by hand (so you can see it has no magic)

```
import numpy as np


def cosine_similarity(vector_a, vector_b):
    """Return how similar two embeddings are, from 0 (unrelated) to 1 (same).

    The idea: measure the angle between the two vectors. We do that by
    taking their dot product and dividing by their lengths. You will
    normally let a library do this -- we write it out once for clarity.
    """
    a = np.array(vector_a)
    b = np.array(vector_b)
    dot = np.dot(a, b)                      # how much they point the same way
    length_a = np.linalg.norm(a)            # length of arrow a
    length_b = np.linalg.norm(b)            # length of arrow b
    return dot / (length_a * length_b)      # angle -> score from 0 to 1
```

Why cosine and not plain distance? Because cosine cares about *direction*, not *size*. A long piece of text and a short piece of text about the same topic point in the same direction even if one arrow is longer; cosine sees them as similar, which is what we want. This is why cosine similarity is the usual default for comparing text embeddings.

SECTION 3

Dimensions: why an embedding has hundreds of numbers

Each number in an embedding is one "dimension." Here is what that means, why more can be better, and why more also costs more.

In Figure 1 we drew the meaning map flat, with just two directions, because that is all we can draw on paper. But a real embedding is not two numbers — it is hundreds or thousands. Each of those numbers is called a **dimension**. Why so many?

ANALOGY — DESCRIBING A PERSON

Suppose you wanted to place every person you know onto a map so that similar people sit close together. With only two facts — say, height and age — you could place them, but you would lump together people who are nothing alike beyond those two traits. Add more facts — friendliness, sense of humour, taste in music, profession — and your placement gets far more accurate, because you are capturing more of what makes each person distinctive. Each new fact is a new dimension. Meaning works the same way: more dimensions give the model more room to capture the many shades of what a piece of text is about.

So more dimensions means more capacity to capture nuance, and that is genuinely useful. But — and this is the part teams get wrong — more dimensions is not simply better. It comes with three real costs.

Storage. Every dimension is a number you have to store for every single chunk. Double the dimensions and you roughly double the storage your vector database needs. Across millions of chunks, that is a real bill.

Speed. Comparing two embeddings means comparing every dimension. More dimensions means more work per comparison, which means slower search, especially at scale.

Diminishing returns. Past a certain point, the extra dimensions stop capturing useful meaning for your particular documents and start capturing noise. You pay for storage and speed but get little or no improvement in answer quality.

> **THE HONEST GUIDANCE ON DIMENSIONS**
>
> More dimensions is a trade-off, not an upgrade. For most document-question-answering work, an embedding with somewhere around a few hundred to about 1,500 dimensions is the practical sweet spot. Reach for a larger model only when you have actually measured that it improves answers *on your own data* — not because a bigger number feels more powerful. MEDIUM CONFIDENCE — DEPENDS ON YOUR DATA

One useful modern detail worth knowing: some embedding models let you keep only the first part of the embedding — say, the first 256 numbers out of 1,024 — and still get most of the quality, which lets you trade a little accuracy for a lot of savings on storage and speed. If you ever need to shrink costs, check whether your model supports this before switching models entirely. MEDIUM CONFIDENCE — MODEL-DEPENDENT

LAB 3.2 — Embed our chunks and look at the dimensions

```
# Import an embedding model from a public library. As always, verify
# the exact import path against the version you installed.
from langchain_openai import OpenAIEmbeddings


def make_embedder():
    """Create one embedding model that we will reuse everywhere.

    Reusing a single embedder is not a style choice -- it is required.
    Section 5 explains why mixing models silently breaks everything.
    """
    return OpenAIEmbeddings(model="text-embedding-3-small")


embedder = make_embedder()

# Turn one chunk into an embedding and inspect it.
chunk_text = "Run a descaling cycle every two months."
vector = embedder.embed_query(chunk_text)

print(f"This embedding has {len(vector)} dimensions")  # e.g. 1536
print(f"First few numbers: {vector[:5]}")              # a peek at the coordinates
```

SECTION 4

See it work: scoring a question against our chunks

Let us put Sections 1 to 3 together and watch a real question score against the pieces of our manual.

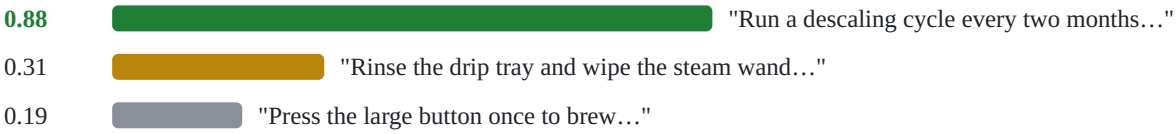
Here is the payoff. We take three chunks from our coffee-machine manual, embed each one, embed a user's question, and then use our cosine function to score how similar the question is to each chunk. The chunk with the highest score is the one we would hand to the model as the answer source.

LAB 3.3 — Score a question against several chunks (our own walkthrough)

```
# Three chunks from our manual, plus the user's question.
chunks = [
    "Run a descaling cycle every two months using descaling solution.",
    "Rinse the drip tray and wipe the steam wand after each use.",
    "Press the large button once to brew a single cup.",
]
question = "How often should I descale the machine?"

# Embed everything with the SAME embedder (this matters -- Section 5).
chunk_vectors = embedder.embed_documents(chunks)   # a list of embeddings
question_vector = embedder.embed_query(question)   # one embedding

# Score the question against each chunk and sort best-first.
scored = []
for text, vector in zip(chunks, chunk_vectors):
    score = cosine_similarity(question_vector, vector)
    scored.append((score, text))

for score, text in sorted(scored, reverse=True):
    print(f"{score:.2f}  {text}")
```

When you run this, you get scores that look something like the picture below. These exact numbers are illustrative — your real scores will differ — but the *shape* is always the same: the chunk that truly answers the question scores clearly highest, and the unrelated chunks score lower.

ILLUSTRATIVE RESULT for "How often should I descale the machine?"



The descaling chunk wins by a wide margin. Notice the **gap** between the top score and the rest — that gap is a signal we will use later: a clear winner means we found a real answer; if every score were low and similar, it would mean the answer probably is not in our documents at all.

That last point is worth holding onto for the debugging module. The scores are not just for ranking — they also tell you how *confident* the match is. A high top score with a clear gap means "I found it." A batch of uniformly low scores means "this question is probably not answered anywhere in my documents," which is exactly when a good system should say "I do not know" instead of inventing an answer.

SECTION 5

The one rule you must never break

Use the same embedding model for your documents and your questions. Always. Here is why breaking it fails silently.

This is the single most important rule about embeddings, and it causes more confusing failures than almost anything else, because when you break it nothing crashes — you just get quietly useless results.

Remember that an embedding model places text onto a meaning map. The catch is that **every model builds its own private map.** Model A's coordinates and Model B's coordinates are in completely different worlds. A point at coordinates "(120, 105)" on Model A's map has nothing to do with "(120, 105)" on Model B's map. So if you embed your documents with Model A but embed your questions with Model B, you are measuring distances between two maps that share no common ground. The numbers still compute — no error appears — but the result is meaningless, and your retrieval returns near-random pieces.

ANALOGY — TWO DIFFERENT CITIES, SAME STREET NAMES

Imagine giving someone directions using street names from London while they are standing in New York. Both cities have a "Broadway" and a "Park Lane," so the instructions *sound* valid and nobody throws an error — but following them lands you nowhere near where you meant. Two embedding models are two different cities. You cannot navigate one using the other's map.

THE RULE, AND ITS CONSEQUENCE

Pick one embedding model and use it for both indexing and querying. If you ever decide to switch models, you must re-embed your *entire* document collection with the new model — there is no shortcut, because the old embeddings live on the old map. (This is the same costly "re-index trap" we flagged among the production failures: choosing your model carefully up front saves you an expensive re-do later.) HIGH CONFIDENCE

SECTION 6

Choosing an embedding model

There is no single best model. There is a best model for your constraints. Here is how to decide.

Embedding models come in two broad kinds. The first is a **hosted model**, which you use over the internet by sending text to a company's service and getting embeddings back. The second is a **self-hosted open model**, which you download and run on your own computers. Neither is simply better; they trade off different things, and the right choice depends on what you care about most.

What you care about	Hosted model (used over the internet)	Self-hosted open model (you run it)
Ease of setup	Very easy — call a service, done.	Harder — you manage the machines that run it.
Cost at large scale	You pay per use; this adds up as volume grows.	Mostly fixed compute cost; usually cheaper at high volume.
Data privacy	Your text leaves your network to be embedded.	Your text never leaves your own systems.
Quality	Generally strong and well-maintained.	The best open models are now very competitive.
Speed	Includes an internet round-trip each time.	Runs locally; can be faster once set up.

A reasonable way to choose, in plain terms: if you are getting started, value simplicity, and your data is not sensitive, a good hosted model is the fastest path and a perfectly sound choice. If your data must stay private (medical, legal, confidential business documents), or if you will run an enormous volume where per-use costs would pile up, a self-hosted open model becomes more attractive. MEDIUM CONFIDENCE — THE DETAILS SHIFT AS MODELS IMPROVE

> **HOW TO ACTUALLY DECIDE — MEASURE, DO NOT GUESS**
>
> Whatever model you are tempted by, the honest test is the same: build a small set of real questions from your own domain, embed your real documents with the candidate model, and check whether the right chunks come back. A model that tops a public leaderboard can still do worse on *your* specific documents. The leaderboard is a hint; your own data is the judge.

SECTION 7

Faults and corrections: the embedding bugs you will hit

For each fault we show the broken code, explain plainly why it breaks, then show the correction.

FAULT 1 — Embedding documents and questions with different models

THE BROKEN CODE

```
doc_embedder = OpenAIEmbeddings(model="text-embedding-3-small")
query_embedder = OpenAIEmbeddings(model="text-embedding-3-large")  # different!

chunk_vectors = doc_embedder.embed_documents(chunks)
question_vector = query_embedder.embed_query(question)   # different map
```

WHY IT BREAKS

As Section 5 explained, each model builds its own meaning map. Embedding documents on one map and questions on another means you are comparing coordinates from two unrelated worlds. Nothing errors; retrieval just quietly returns near-random pieces, and you waste days blaming the prompt or the database.

THE CORRECTION

```
embedder = OpenAIEmbeddings(model="text-embedding-3-small")  # one model

chunk_vectors = embedder.embed_documents(chunks)
question_vector = embedder.embed_query(question)   # same map, every time
```

Define the embedder once and reuse it everywhere, exactly as our `make_embedder` helper does.

FAULT 2 — Embedding one chunk at a time

THE BROKEN CODE

```
# Calling the model separately for every single chunk.
vectors = []
for chunk in chunks:
    vectors.append(embedder.embed_query(chunk))   # one slow call each
```

WHY IT BREAKS

Each call to a hosted model carries a fixed overhead (a network round-trip, a bit of waiting). Doing it once per chunk means thousands of slow, separate calls when you index a large collection. It is far slower and, for paid models, can cost more. The fix is to send chunks in batches so the overhead is shared.

THE CORRECTION

```
# embed_documents handles many texts in efficient batches for you.
vectors = embedder.embed_documents(chunks)   # one batched operation
```

Use the batch method (`embed_documents`) for many chunks, and save the single-item method (`embed_query`) for the one-off question at query time.

FAULT 3 — Re-embedding everything on every run

THE BROKEN CODE

```
# Every time the script runs, it re-embeds all chunks from scratch,
# even the ones that have not changed -- paying again each time.
vectors = embedder.embed_documents(all_chunks)
```

WHY IT BREAKS

Embedding costs time and, for hosted models, money. If your pipeline re-embeds the entire collection on every run — including chunks that have not changed since last time — you pay that full cost repeatedly for no reason. On a large collection this is a slow, expensive habit that hides until the bill arrives.

THE CORRECTION

```
# Remember embeddings by the content that produced them, so unchanged
# chunks are looked up instead of re-embedded. (Same stable-ID idea
# we met in Module 2.)
import hashlib

cache = {}   # in real life this is a file or a database

def embed_with_cache(text):
    key = hashlib.sha256(text.encode()).hexdigest()
    if key not in cache:                        # only embed if we have not seen it
        cache[key] = embedder.embed_query(text)
    return cache[key]
```

Cache embeddings keyed by the chunk's content. Re-embed only what actually changed.

FAULT 4 — Reading the similarity score the wrong way round

THE BROKEN CODE

```
# Some tools return a DISTANCE (lower = closer), not a similarity.
# Here we wrongly assume higher is always better and keep the worst match.
best = max(results, key=lambda r: r.distance)   # picks the FARTHEST piece
```

WHY IT BREAKS

Some tools give you a similarity (where higher means closer) and others give you a distance (where lower means closer). If you assume the wrong direction, you systematically pick the *least* relevant pieces while believing you picked the best, and the system feels broken for no obvious reason.

THE CORRECTION

```
# Check which one your tool returns, then sort accordingly.
# If it returns distance (lower = closer):
best = min(results, key=lambda r: r.distance)   # closest piece
# If it returns similarity (higher = closer), use max() instead.
```

Always confirm whether your tool reports similarity or distance before you threshold or sort on it. A two-minute check prevents a baffling bug.

FAULT 5 — Feeding empty or oversized text to the embedder

THE BROKEN CODE

```
# Some chunks are empty (blank pages) or far longer than the model
# can read. Sending them as-is causes errors or silent truncation.
vectors = embedder.embed_documents(raw_chunks)
```

WHY IT BREAKS

Embedding models reject empty text and can only read up to a maximum length. Blank chunks (often from blank pages or over-aggressive cleaning) cause errors, and over-long chunks get silently cut off, so part of their meaning is lost from the embedding without any warning.

THE CORRECTION

```
# Guard the inputs: drop empties and flag anything suspiciously long.
def safe_chunks(chunks, max_chars=8000):
    cleaned = []
    for text in chunks:
        text = text.strip()
        if not text:                # skip empty pieces
            continue
        if len(text) > max_chars:   # too long -> your chunking is off
            print(f"Warning: a chunk is {len(text)} chars; revisit Module 2.")
        cleaned.append(text)
    return cleaned

vectors = embedder.embed_documents(safe_chunks(raw_chunks))
```

An over-long chunk here is usually a sign your chunking in Module 2 needs another look — the two modules are connected.

Checkpoint: try it yourself

Embeddings click the moment you watch them score real text. This short exercise takes about twenty minutes and makes the idea concrete.

1. Take five short sentences of your own. Make three of them about one topic (say, cleaning something) and two about a different topic (say, cost or time). Write them down before you embed anything, so you have a prediction to check against.
2. Embed all five with a single embedding model, then use the `cosine_similarity` helper from Lab 3.1 to score every sentence against every other sentence.
3. Look at the scores. Do the three same-topic sentences score higher with each other than with the two off-topic ones? They should — and seeing your prediction confirmed is the moment embeddings stop being abstract.
4. Now write a question about your first topic, embed it, and score it against all five sentences. Check that the same-topic sentences rise to the top. That is retrieval, in miniature, with your own hands.

> **WHAT YOU SHOULD NOTICE**
>
> Same-topic sentences cluster together with higher scores, just like Figure 1 promised — even when they share no words in common. That is the whole point of embeddings: they match on *meaning*, not on matching words. Notice too that the scores are never a perfect 1 or 0; meaning is a matter of degree, and the scores reflect that.

What to carry into Module 4

> **THE SIX THINGS TO REMEMBER**
>
> **One,** an embedding turns text into a list of numbers that act like coordinates on a "meaning map," so similar meanings land close together. **Two,** once text is points in space, "find the relevant piece" becomes "find the nearest points," usually measured by cosine similarity. **Three,** each number is a dimension; more dimensions capture more nuance but cost more storage, more speed, and eventually give diminishing returns. **Four,** the similarity score also signals confidence — a clear top score means a real match, uniformly low scores mean the answer may not be there. **Five,** never break the one rule: the same model must embed both documents and questions, or you compare two unrelated maps. **Six,** there is no single best model — choose on setup, cost, privacy, and speed, and let your own data be the judge.

We can now turn any piece of text into a point on the meaning map, and we can measure how close two points are. But doing that comparison against millions of chunks, fast, needs a special kind of storage built for the job. In Module 4 we meet the vector database — the home for all these embeddings, and the tool that finds the nearest points in milliseconds instead of minutes.